

Open in app ↗

Sign up

Sign in

Medium

Search

Sign in to Medium with
Google

Continue

Cpp



React



Raytracing



Ray Tracing with C++

On my mission to prove why doom scrolling is better than working in C++

10 min read · Nov 28, 2025



Sudeep Ranjan Sahoo

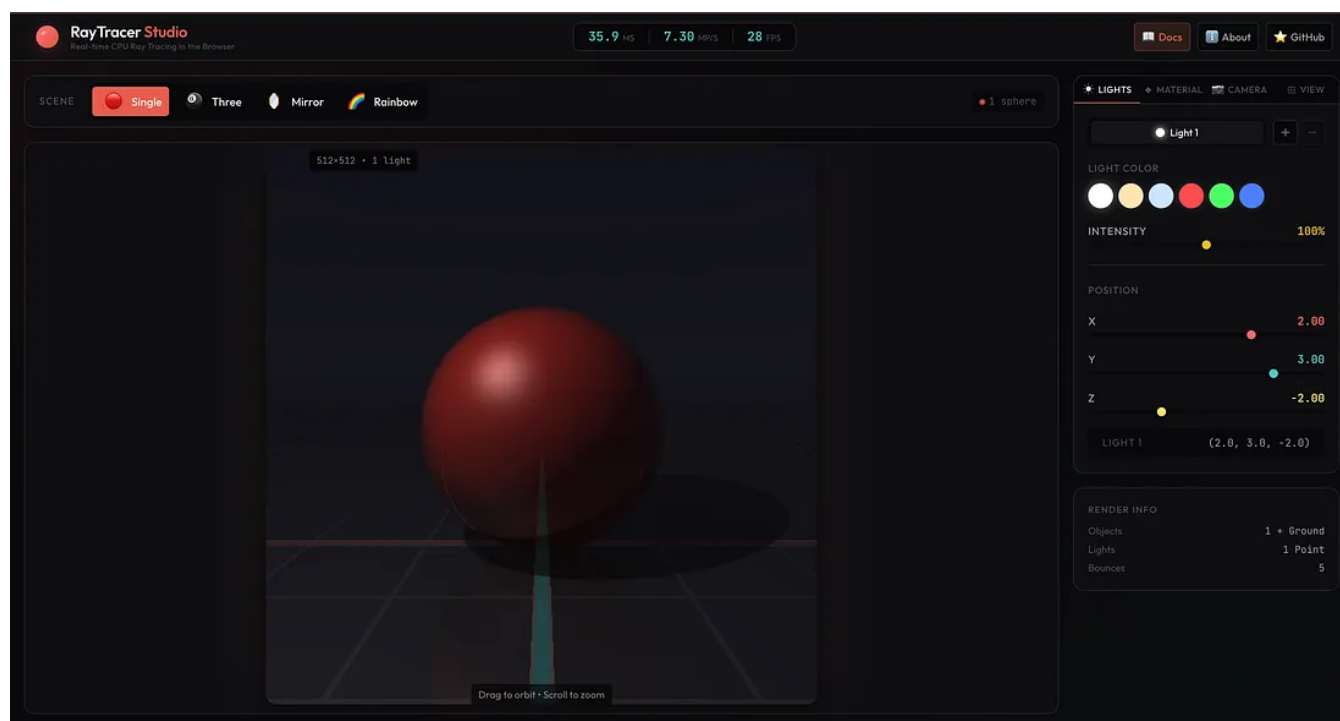
Follow



Listen



Share



RayTracer studio: The client part for the C++ based raytracing experiment.

1. Introduction: Why I Decided to Raytrace My Sanity

There I was, doom-scrolling my way through Instagram reels like any self-respecting procrastinator, when I stumbled across this one clip that genuinely made

me stop.

It was a behind-the-scenes look at Pixar, where the team spent literal months creating a *realistic water scene* for an animated film.

I remember thinking, “Wait, *all that* just to make water look... wet?”

It blew my mind.

That was my rabbit hole moment.

I started diving deeper and learned they use this magical thing called **ray tracing**, the secret sauce behind those stunning reflections, soft shadows, and yep, even that beautifully animated water.

Being the kind of person who occasionally chooses pain by writing C++ (because who doesn’t enjoy chasing segfaults at 2 AM?), I thought:

Why not try this myself?

Also, I was kind of bored with CRUD apps.

I wanted to explore something different, something that felt like building a tiny universe where light has rules and objects cast shadows (plus, let’s be honest, it sounds *way cooler* when you tell people you’re working on a ray tracer). So, guess what, I went into chai mode again and left my 3 am anime binge watch to start working on this...



My life lately

NOTE: I have not written complete technical details here anyway because then you will be bored to hell. It just contains the summary of it and what my approach was. To know the complete details technically, the code and the scripts etc, you can read the docs here :

Code: <https://github.com/srs-sudeep/rayTracing> (Please don't forget to give it a star)

Web portal: <https://ray-tracing.vercel.app/>

Docs: <https://ray-tracing.vercel.app/docs>

2. So, What Is Ray Tracing Anyway?

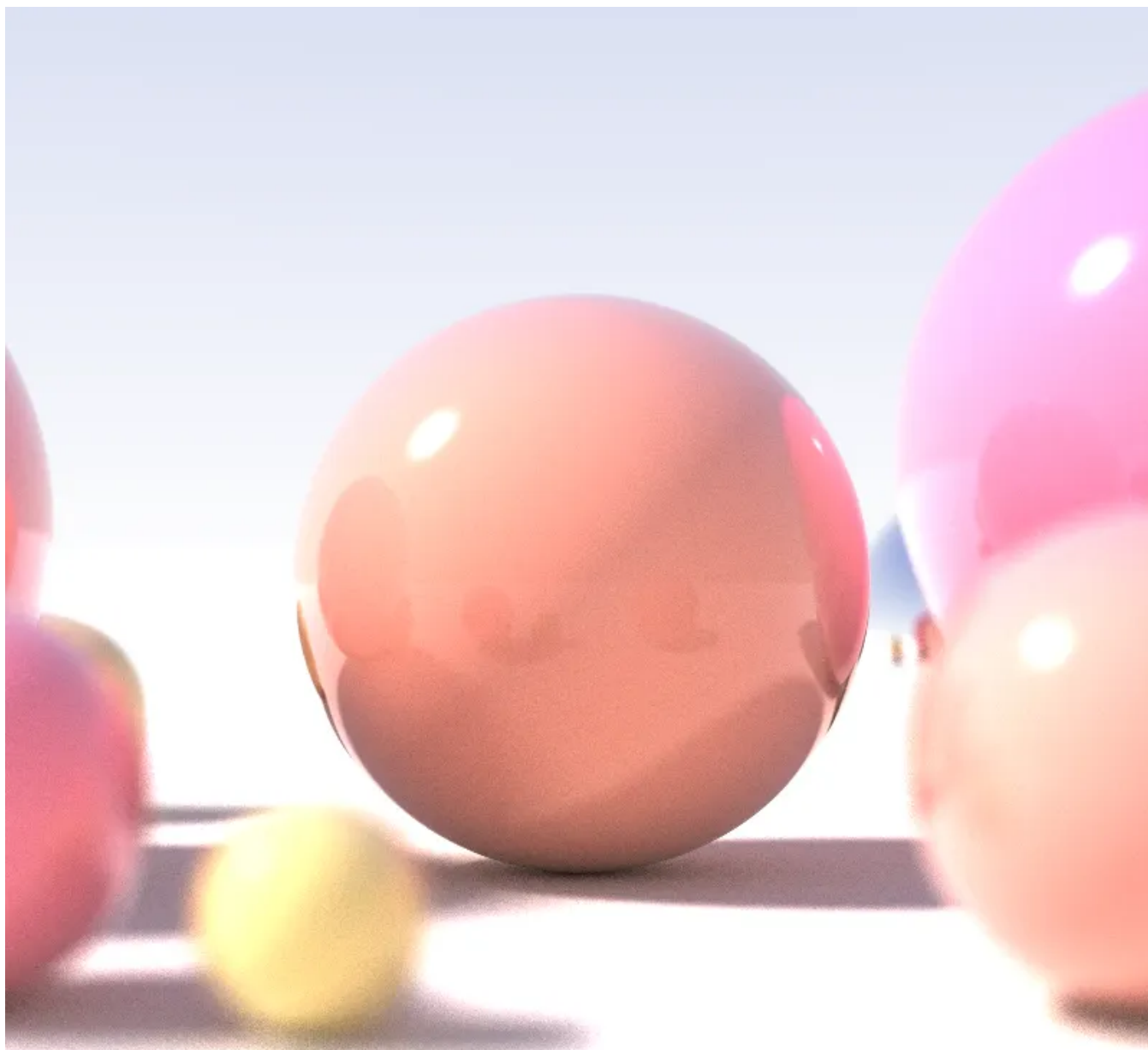
Ray tracing is like teaching your computer to be a very slow, but very accurate, **light detective**.

Instead of painting pixels directly, you send out imaginary rays from the camera and let them bounce around your scene. They might hit a sphere, reflect off a shiny surface, get absorbed, or maybe miss everything and hit the void (sounds like some of my life choices).

Real-world uses?

- **Movies & Animation:** Pixar, Marvel, all those pretty CGI explosions.
- **Video Games:** Real-time ray tracing in modern games for life-like lighting and reflections.
- **Scientific Simulations:** Optics, medical imaging, and more.

It's everywhere. But to us, it's mostly about **having fun with math and making pretty pictures**.



An image describing rayTracing (Source Wikipedia)

3. My Approach: Step by (Painful) Step

I wanted to keep things simple, build everything from scratch, and understand every line of code instead of using a pre-made package or library. I wanted to explore what actually runs in the background, what makes things run. I came across

the best resource for the same <https://raytracing.github.io/> by Steve Hollasch, Trevor D Black and Peter Shirley. Following their book, I started working on what I should make, which turns out to be something like this:

The Goal: Create a ray tracer in C++ that somehow runs in a web browser alongside a modern React UI.

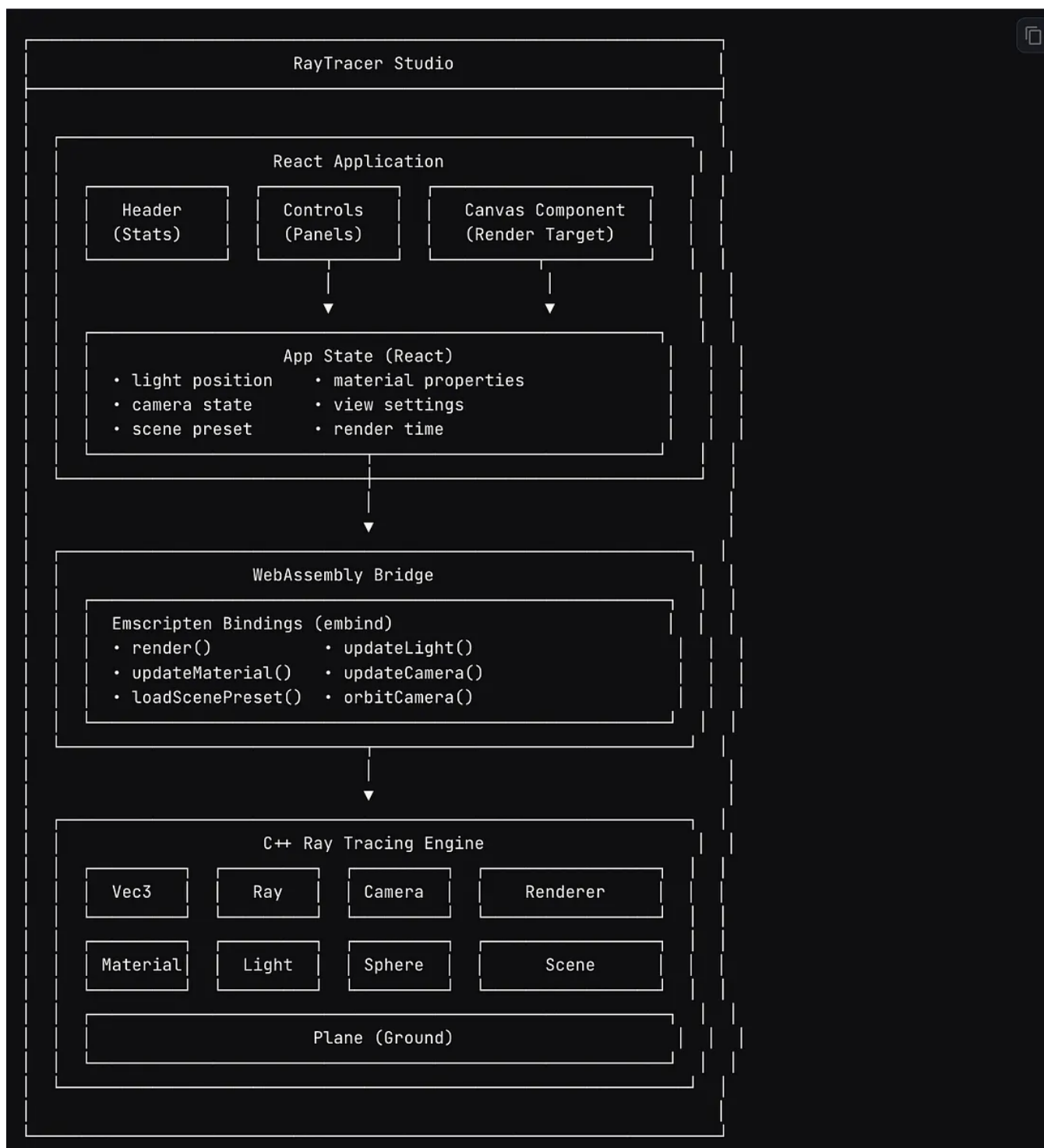
The Problem: Browsers don't speak C++. They speak JavaScript. And JavaScript looks at C++ the way cats look at water with deep suspicion and a strong desire to stay away.

The Solution: *WebAssembly*

Enter Emscripten: The Bridge Between Worlds

Emscripten is this magical tool that compiles C++ code into **WebAssembly** a binary format that browsers can run at near-native speeds. It's like Google Translate, but for programming languages, and it actually works.

Here's the beautiful architecture I ended up with:



Complete thousand feet view of my project on how RayTracerStudio will work.

So now I had:

The Brain (C++):

Handles all the heavy computation — ray physics, intersections, shading, and core

math.

The Translator (Emscripten):

Converts the C++ engine into something the browser can run.

The Speed (WebAssembly):

Executes the C++ code inside the browser at near-native performance.

The Pretty Face (React):

Provides the polished UI — sliders, buttons, and user controls.

The Canvas (HTML Canvas):

The surface where the final rendered pixels appear.

◆ Step 1: Just Print the Pixels

I started with the most basic task: **writing a PPM image file**.

A simple format where you can print RGB values into a text file, and boom image.

Then I decided to print an Italian flag (because mamma mia, why not??).

You can run the below to generate the image in ppm and then use tools like **ImageMagick** to generate the image

```
#include <iostream>
#include <cmath>

int main() {
    int width = 600;
    int height = 400;

    std::cout << "P3\n" << width << ' ' << height << "\n255\n";

    for (int y = 0; y < height; ++y) {
        for (int x = 0; x < width; ++x) {
            double red = 0.0, green = 0.0, blue = 0.0;
```

```
    if (x < width / 3) {
        red = 0.0;
        green = 146.0 / 255.0;
        blue = 70.0 / 255.0;
    } else if (x < 2 * width / 3) {
        red = 1.0;
        green = 1.0;
        blue = 1.0;
    } else {
        red = 206.0 / 255.0;
        green = 43.0 / 255.0;
        blue = 55.0 / 255.0;
    }

    int ir = static_cast<int>(255.999 * red);
    int ig = static_cast<int>(255.999 * green);
    int ib = static_cast<int>(255.999 * blue);

    std::cout << ir << ' ' << ig << ' ' << ib << '\n';
}
}
```

Note: I have also added the script render.bat and render.sh to run the above and generate the PNG format of the image generated by the above using magick tools

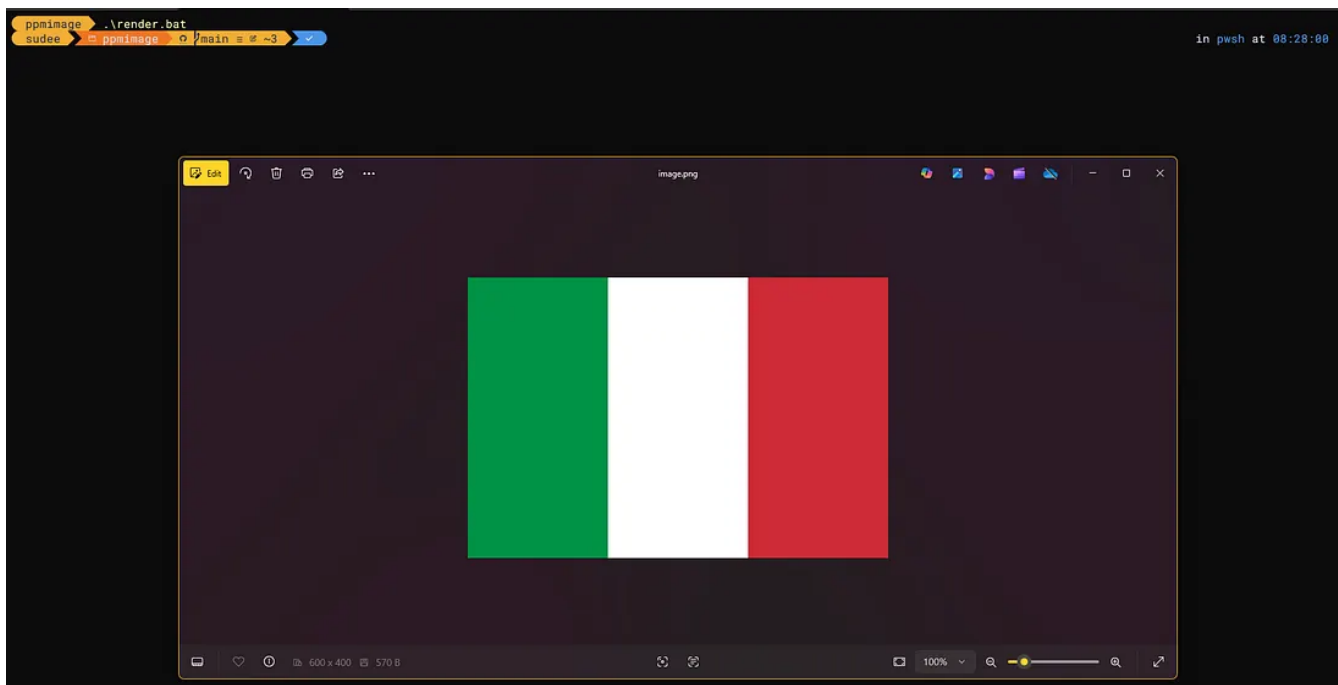


Image of Italy Flag generated using the above code.

♦ Step 2: Drawing a Circle (or Trying To)

My first goal was simple: get a sphere on screen.

“How hard can it be?” I thought, with the hubris of someone who had clearly never read a graphics programming book.

Turns out, MATH. So much math.

To know if a ray hits a sphere, you need to solve the **quadratic formula**. Yes, that thing from high school that you were sure you’d never use again

A sphere is all points where: $|P - C|^2 = r^2$

Substitute ray equation $P = O + tD$:

$$|O + tD - C|^2 = r^2$$

This becomes:

$$at^2 + bt + c = 0$$

where:

$$a = D \cdot D$$

$$b = 2D \cdot (O - C)$$

$$c = (O - C) \cdot (O - C) - r^2$$

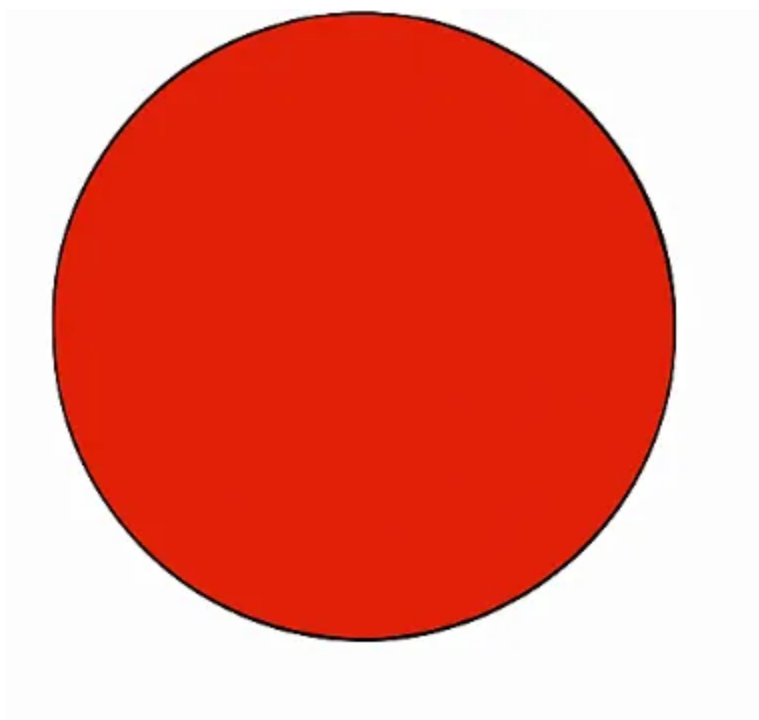
I stared at this formula for approximately 47 minutes before it clicked. Then I felt like a god.

```
float discriminant = b*b - 4*a*c;

if (discriminant < 0) {
    // Ray missed! No intersection
} else {
    float t = (-b - sqrt(discriminant)) / (2*a);
    // WE HIT SOMETHING! t is how far away it is
}
```

My First Render

I remember the exact moment. I had written about 300 lines of C++, compiled it to WebAssembly for the 47th time, refreshed the browser, and...



Umm why not a RED Circle

A RED CIRCLE.

Okay, it looked more like a gradient blob with existential issues, but it was MY blob. I had traced rays. Through pixels. Into a mathematical sphere. And colors came out.

I may have yelled “IT WORKS!” loud enough for my neighbors to wonder if I’d finally

lost it.

◆ Step 3: Let There Be Light (And Shadows)

A flat-colored circle is cool for about 3 seconds. Then you want lighting.

Enter the **Blinn-Phong shading model** — named after Jim Blinn and Bui Tuong Phong, two graphics legends who probably never had to debug WebAssembly bindings at 2 AM.

The model combines three types of lighting:

1. **Ambient Light** — That baseline illumination so shadows aren't pure black

```
Vec3 ambient = color * 0.1f; // Just a little light everywhere
```

2. **Diffuse Light** — How surfaces scatter light (the “matte” look)

```
float diffuse = max(0, N · L); // N = surface normal, L = light direction
```

3. **Specular Highlights** — Those shiny spots that make things look... shiny

```
Vec3 H = normalize(L + V); // Half-vector magic  
float specular = pow(max(0, N · H), shininess);
```

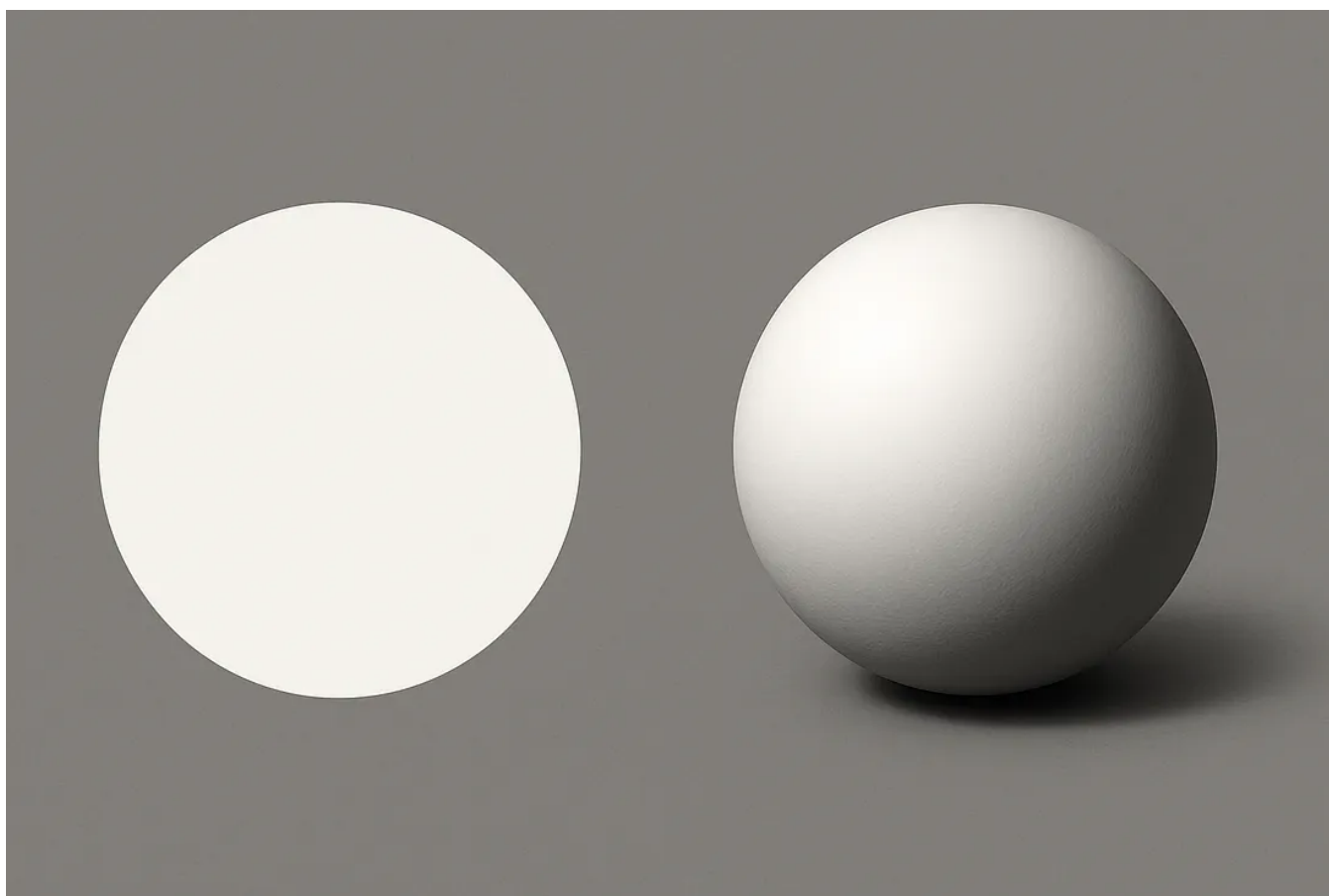


Image of sphere with a shadow and light

And then came **SHADOWS!**

Get Sudeep Ranjan Sahoo's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐

Remember me for faster sign in

To check if a point is in shadow, we shoot another ray from that point toward the light. If it hits something on the way? Shadow time.

```
bool isInShadow(Vec3 point, Vec3 lightPos) {
    Vec3 toLight = lightPos - point;
    Ray shadowRay(point + tiny_offset, normalize(toLight));

    // If this ray hits anything before reaching the light...
    for (auto& object : scene) {
        if (object.intersect(shadowRay).t < distance_to_light) {
            return true; // You're in the dark, buddy
        }
    }
    return false;
}
```

♦ Step 4: Mirror, Mirror on the Wall

Here's where things got *really* fun: reflections.

Ray tracing handles reflections beautifully because, well, that's literally what light does — it bounces. So when a ray hits a reflective surface, we just... trace another ray in the reflected direction. Simple, right?

```
Vec3 traceRay(Ray ray, int depth) {
    if (depth >= maxDepth) return background; // Stop the madness

    HitRecord hit = findIntersection(ray);
    if (!hit) return background;

    Vec3 localColor = calculateLighting(ray, hit);

    if (hit.material.reflectivity > 0) {
        // Compute reflection direction
        Vec3 reflectDir = ray.direction.reflect(hit.normal);
        Ray reflectRay(hit.point, reflectDir);

        // RECURSION! Trace the reflected ray
        Vec3 reflectedColor = traceRay(reflectRay, depth + 1);

        // Blend it together
        localColor = mix(localColor, reflectedColor, reflectivity);
    }
}
```

```
    return localColor;  
}
```

The Fresnel Effect: Real Physics Makes Everything Better

Here's a fun fact: real surfaces are more reflective when you look at them at grazing angles. Look at a lake straight down? You see the bottom. Look across it at a shallow angle? Mirror city.

```
float cosTheta = abs(normal.dot(viewDir * -1.0f));  
float fresnelFactor = reflectivity +  
    (1.0f - reflectivity) * pow(1.0f - cosTheta, 3.0f);
```

This single formula made my renders go from “that’s nice” to “okay that’s actually kinda beautiful.”

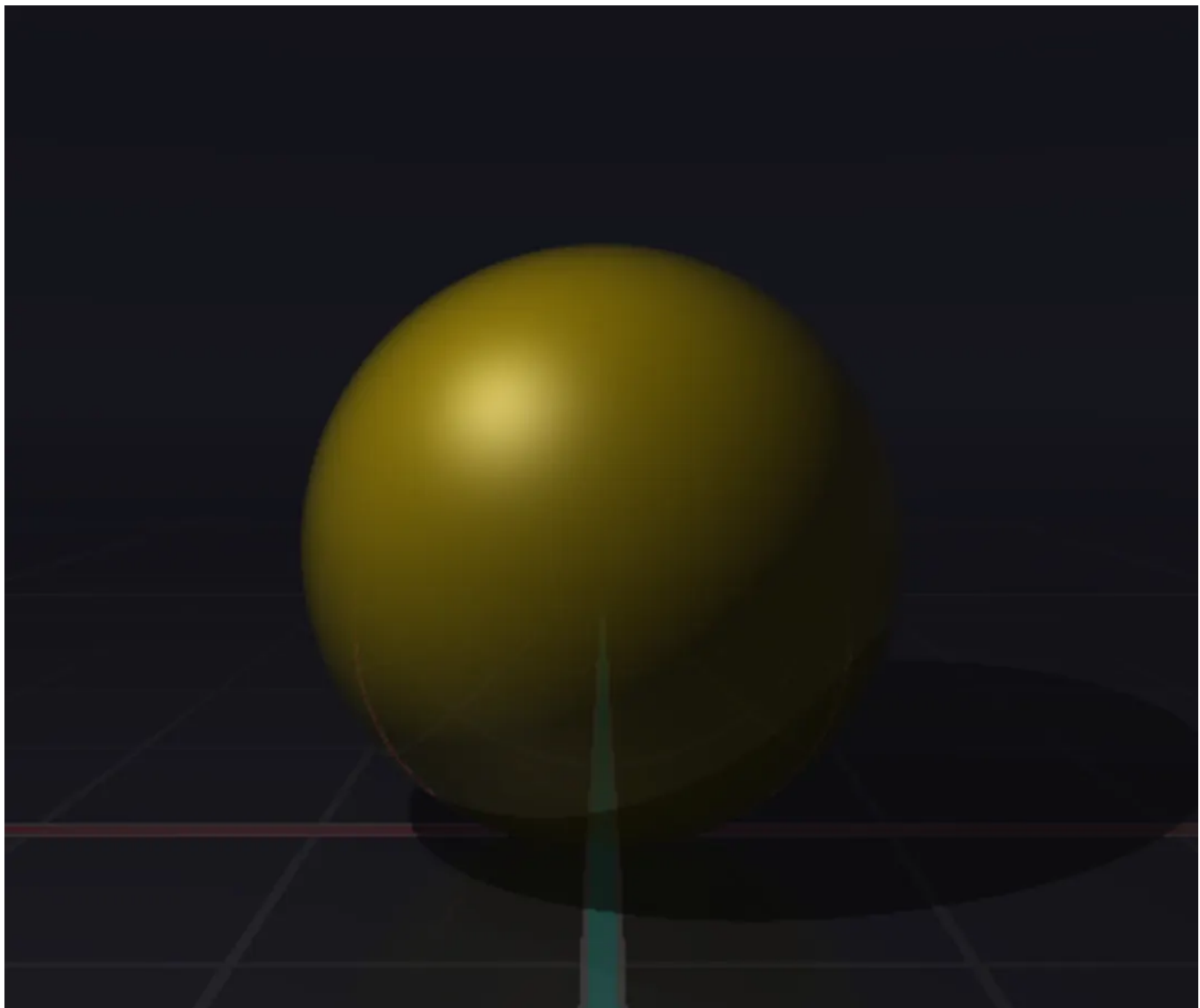
♦ Step 5: Ground Plane & The Blender Grid

At this point, my sphere was floating in a void. Very existential, not very impressive for demos.

Solution: Add a ground plane with a grid!

Inspired by Blender’s default scene, I implemented an infinite plane with:

- ✨ Red X-axis line
- ✨ Teal Z-axis line
- ✨ Major and minor grid lines
- ✨ Distance-based fade (so the grid doesn’t go on forever)
- ✨ Its own reflections (for that polished floor look)



Blender like grid objects with multi configurations.

◆ **Step 6: Multiple Spheres & Scene Presets**

One sphere is lonely. I needed to give it friends.

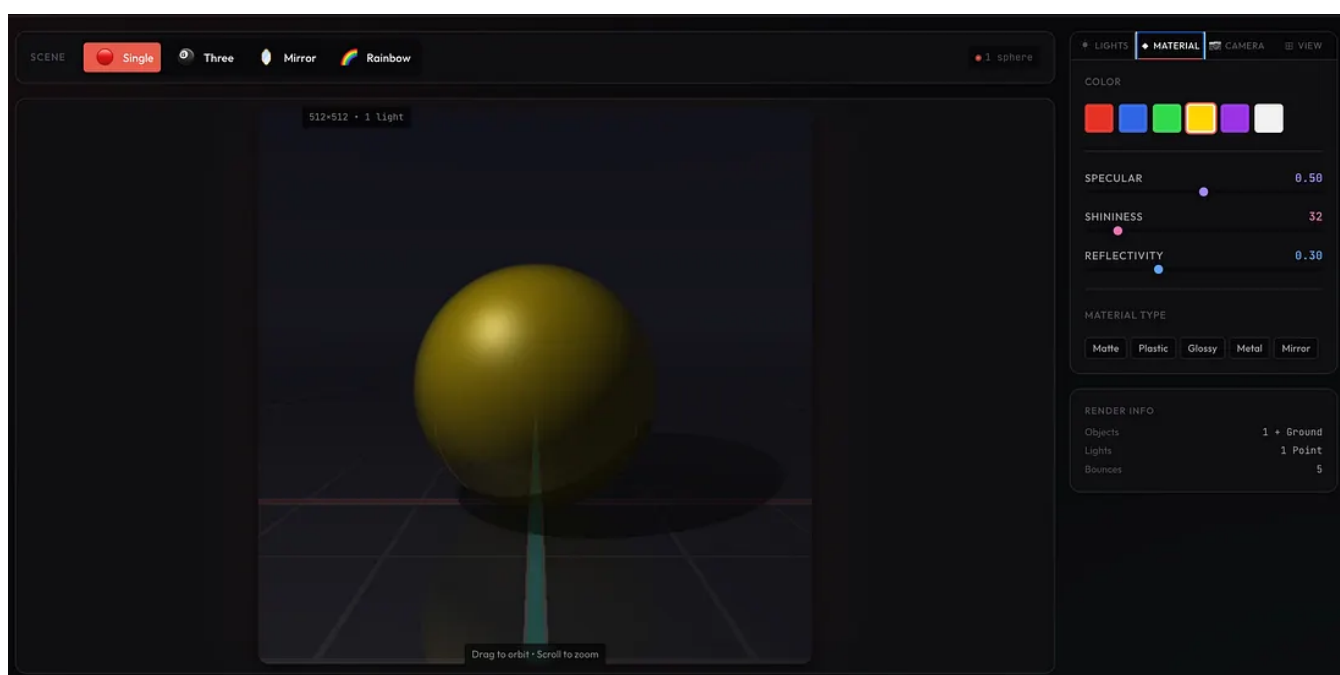
```
std::vector<Sphere> spheres = {  
    Sphere(Vec3(0, 0.5, 0), 0.5, redMaterial),  
    Sphere(Vec3(-1.5, 0.5, 0.5), 0.5, blueMaterial),  
    Sphere(Vec3(1.5, 0.5, 0.5), 0.5, greenMaterial),  
};
```

Now the ray tracer had to check intersections with ALL spheres and find the closest one:

```
HitRecord closestHit = {.t = INFINITY};  
for (auto& sphere : spheres) {  
    HitRecord hit = sphere.intersect(ray);  
    if (hit.t > 0 && hit.t < closestHit.t) {  
        closestHit = hit;  
    }  
}
```

I added scene presets to showcase different configurations:

-  **Single Sphere** — The classic
-  **Three Spheres** — RGB squad
-  **Mirror Scene** — Chrome balls reflecting infinity
-  **Rainbow** — Because why not



Main screen showcasing all the configurations.

4. More about React side

All this C++ code is useless if nobody can interact with it. Enter React.

I built a dark, sleek interface with:

Control Panels:

- **Light Controls** — Move the light around in 3D space
- **Material Controls** — Adjust reflectivity, shininess, color
- **Camera Controls** — Orbit, zoom, change field of view
- **View Settings** — Resolution, grid toggle, max reflection bounces

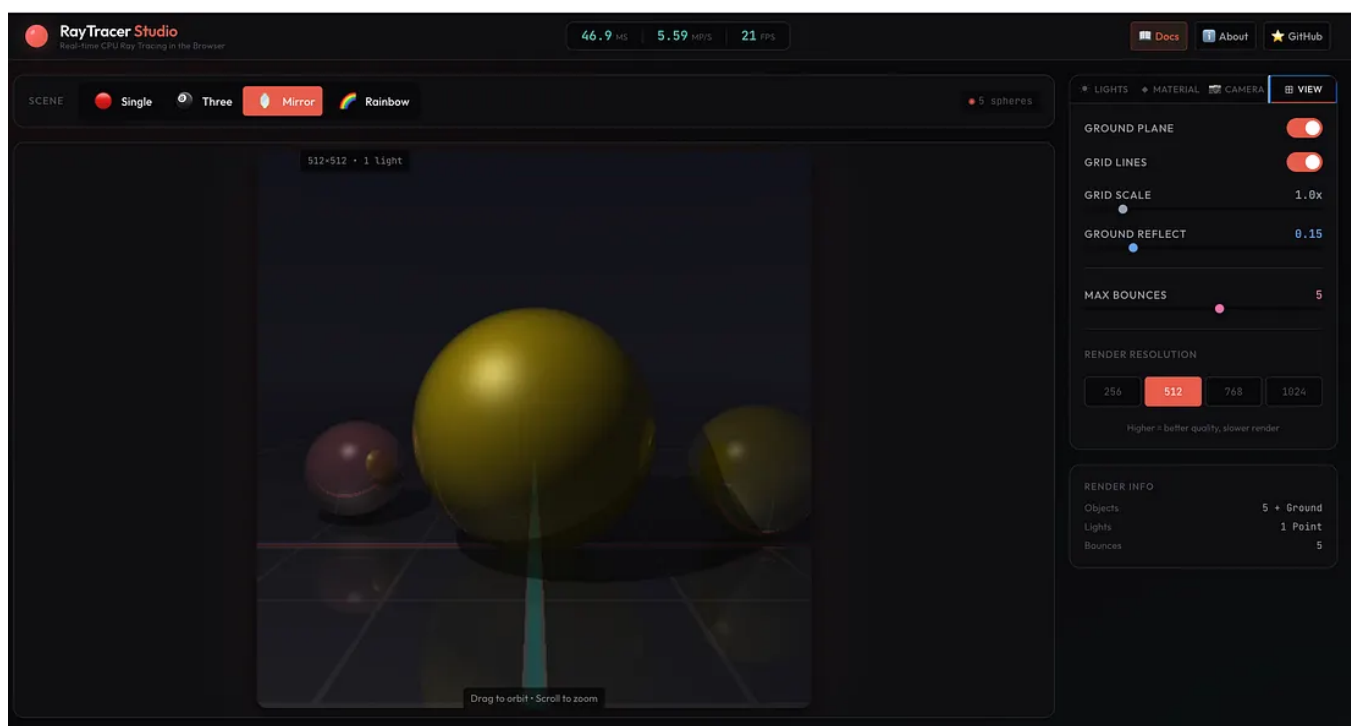
Performance Stats:

- Render time in milliseconds

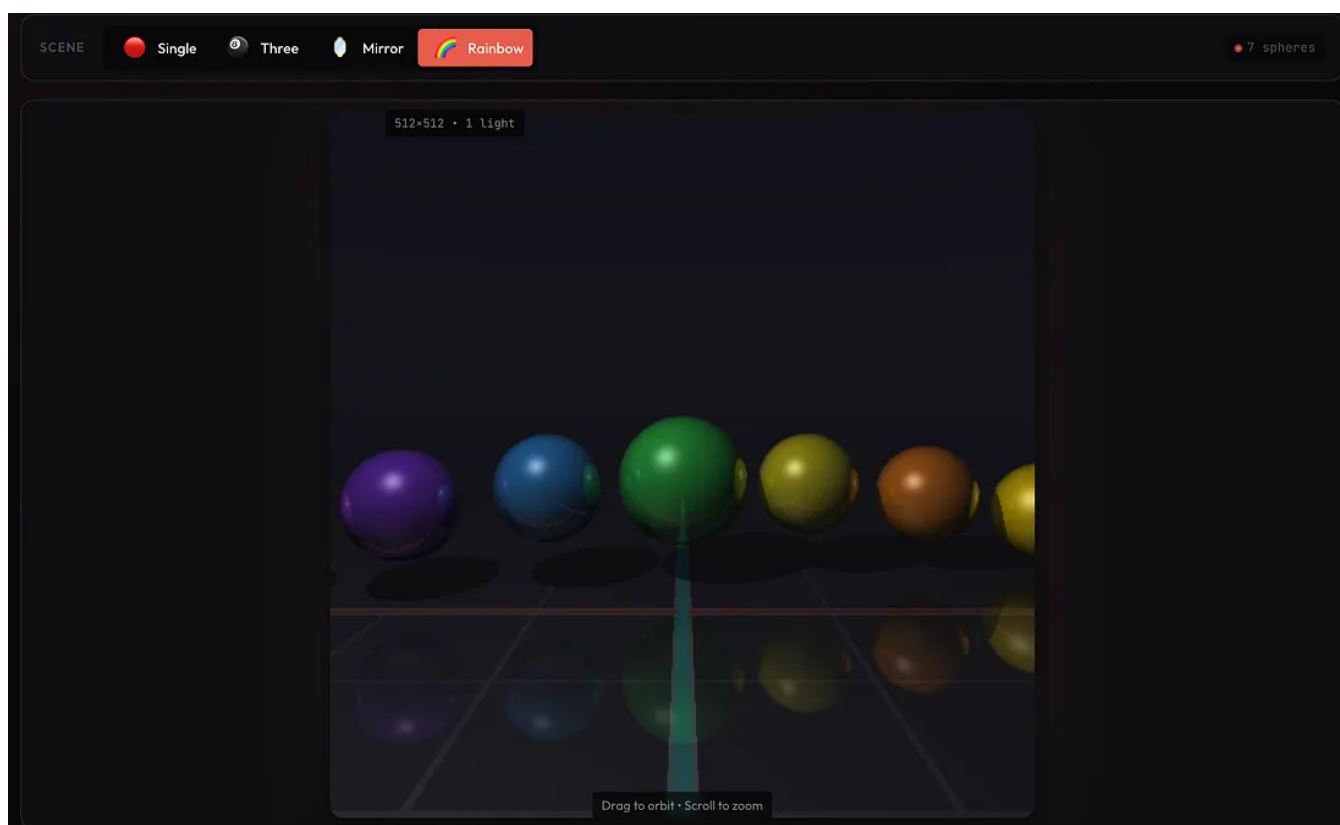
- Megapixels per second
- Actual FPS

Interactive Canvas:

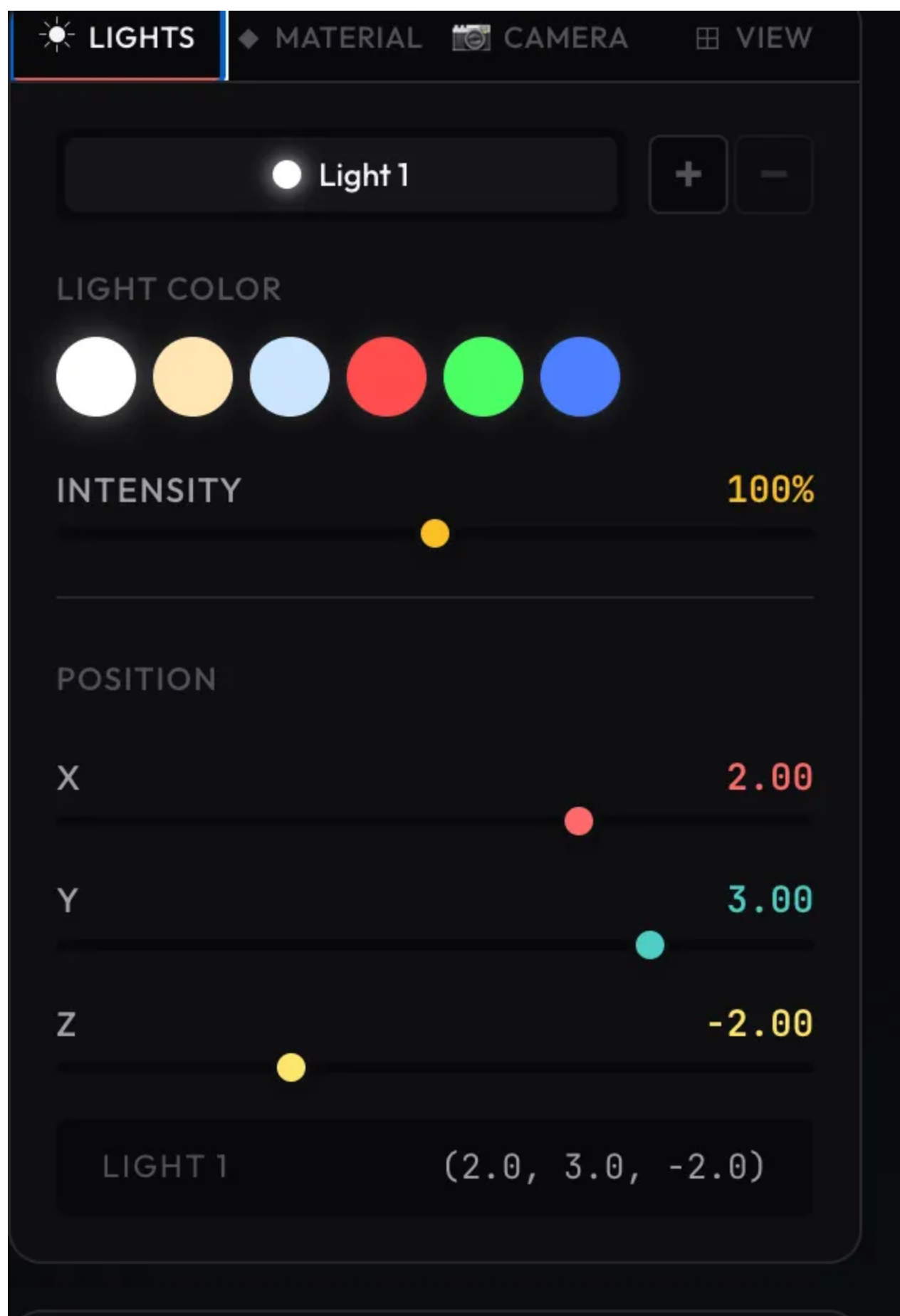
- Mouse drag to orbit camera
- Scroll wheel to zoom
- Click preset buttons to change scenes

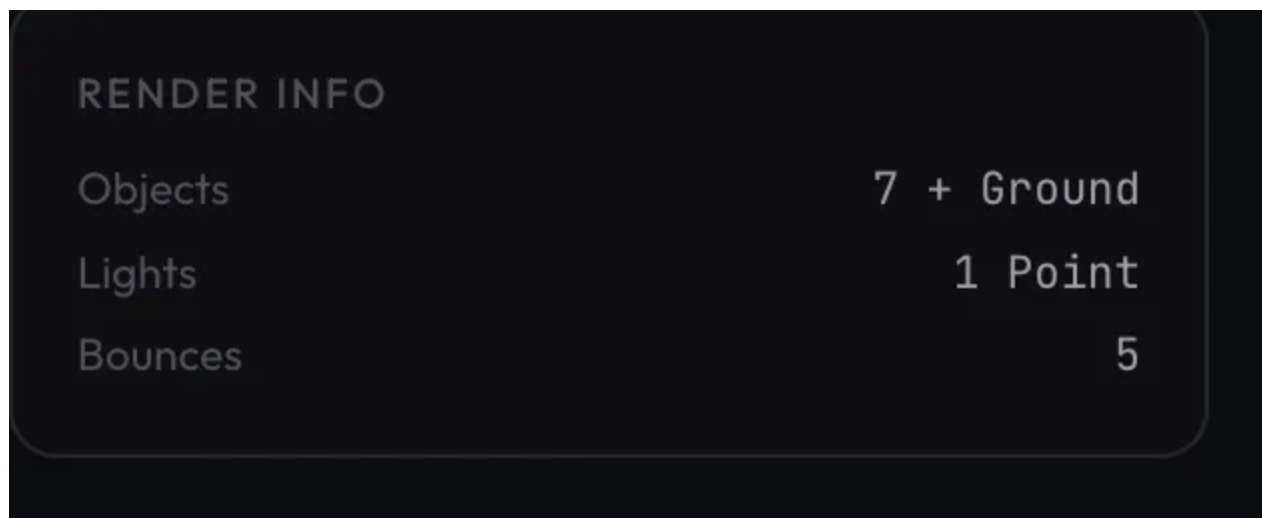


Images showcasing Mirror configuration with options of View opened



Rainbow scene





Configuration Panel

The code to bridge React and WebAssembly looks something like this:

```
const renderFrame = () => {  
  // Update C++ state from React state  
  wasmModule.updateLight(light.x, light.y, light.z);  
  wasmModule.updateMaterial(specular, shininess, reflectivity);  
  
  // Call C++ render function – this is where the magic happens  
  const pixelVector = wasmModule.render(width, height);  
  
  // Copy pixel data to Canvas  
  const imageData = new ImageData(pixelData, width, height);  
  ctx.putImageData(imageData, 0, 0);  
  
  // IMPORTANT: Clean up C++ memory!  
  pixelVector.delete();  
};
```

What I Learned (Besides Ray Tracing)

1. C++ Isn't Dead. It's Everywhere.

Here's what's written in C/C++:

- 🎮 Every AAA game engine (Unreal, Unity's core)
- 💻 Operating system kernels (Windows, Linux, macOS)
- 🌐 Web browsers (Chrome, Firefox, Safari)
- 📱 Mobile apps (Android NDK, iOS low-level)
- 🤖 Machine learning (TensorFlow, PyTorch backends)
- 🎬 Movie rendering (Pixar's RenderMan)
- 🚀 SpaceX flight software (yes, really)

C++ is the stage on which other languages perform.

2. Math Is Actually Cool (Sometimes)

Vector math, quadratic equations, dot products, cross products... Turns out my high school teachers were right. You DO use this stuff.

The moment you see those equations turn into beautiful graphics? Worth every confused hour.

3. Constraints Breed Creativity

By limiting myself to CPU ray tracing (no GPU shaders), I had to deeply understand every optimization. Every pixel earned.

The Live Demo

Enough reading. Go play with it!

 ray-tracing.vercel.app

Move the light around. Make the spheres super reflective. Crank up the resolution. Watch your CPU work for its living.

Need More technical idea?? Check out the docs:

 ray-tracing.vercel.app/docs

Made with ❤️, C++, and an unhealthy amount of caffeine.

— SRS

Cpp

+

React

+

Raytracing

+

Mathematics

+

Webassembly

+



Follow

Written by Sudeep Ranjan Sahoo

7 followers · 6 following

I'm Sudeep Ranjan Sahoo, currently working as Business analyst in Accenture Japan

